Static versus dynamic scheduling on a CPU + GPU workstation

Final benchmark: end-to-end makespan, same cost model on both sides

Real measurements on one laptop (6 P + 7 E workers, one RTX 3050 CUDA worker). Raspberry Pi out of scope.

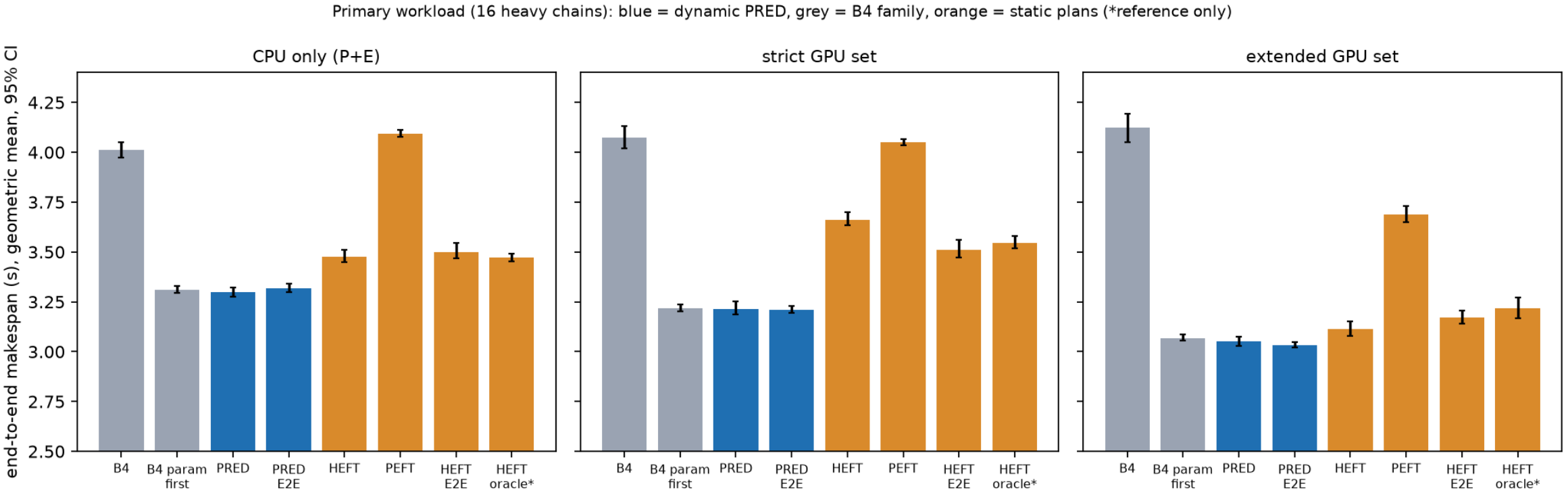
Question and what was built

- Question: is a dynamic scheduler (ranking, late binding, feedback) faster than a fixed static plan from the same cost model - and when?
- Timing fixed: one end-to-end makespan = initialisation + planning + scheduling + execution (old makespan kept for comparison).
- Fair static baselines: HEFT, PEFT, HEFT with end-to-end terms, and an oracle-cost HEFT as a labelled reference.
- New dynamic policy PRED_RANK_GREEDY_E2E (adds artifact I/O and IPC terms; H2D/D2H are already inside the CUDA time, not added twice).
- GPU used only for stages/parameter regions verified in recorded speed-up measurements; one CUDA worker, no extra streams.

Setup

- Workload: image pipeline (source, denoise, edges, features, aggregate), 512 px; chains with light and heavy denoise.
- Primary workload: 16 heavy chains of 64, 25 blocks (24 analysed). Five more workloads of 13 blocks: uniform, 4 heavy, 40 heavy, light 15 and 40 iterations.
- Configurations: CPU only; strict GPU (original verified stages); extended GPU (plus denoise at verified iteration counts).
- Policies: static HEFT / PEFT / HEFT_E2E / oracle reference; dynamic B4, B4_param_first, PRED_RANK_GREEDY, PRED_RANK_GREEDY_E2E.
- Interleaved blocks, mains power and free-RAM checks, output verified for every run; paired bootstrap, Holm over 10 contrasts, +-2% equivalence margin.

Primary result: end-to-end makespan



Extended GPU set: PRED_E2E 3.03 s, PRED 3.05 s, HEFT 3.11 s, B4 4.12 s.

Confirmatory contrasts (Holm family of 10)

id	contrast	effect % [95% CI]	Holm p	classification
C1	PRED vs B4, CPU	-17.8 [-18.8, -16.7]	0.001	faster
C2	PRED_E2E vs PRED, CPU	+0.6 [+0.1, +1.3]	0.141	equivalent (CI inside +-2%)
C3	PRED vs HEFT, CPU	-5.2 [-5.9, -4.6]	0.001	faster
C4	PRED_E2E vs HEFT_E2E, CPU	-5.2 [-5.9, -4.5]	0.001	faster
C5	PRED vs HEFT, strict GPU	-12.2 [-13.4, -11.1]	0.001	faster
C6	PRED_E2E vs HEFT_E2E, strict GPU	-8.5 [-9.7, -7.6]	0.001	faster
C7	PRED_E2E vs PRED, strict GPU	-0.0 [-0.7, +0.5]	0.960	equivalent (CI inside +-2%)
C8	PRED vs B4, strict GPU	-21.2 [-22.6, -19.7]	0.001	faster
C9	PRED strict GPU vs CPU	-2.6 [-3.3, -1.7]	0.001	faster
C10	B4 strict GPU vs CPU	+1.6 [+0.2, +3.0]	0.141	inconclusive

Across workloads (exploratory, unadjusted)

workload (CPU only)	PRED vs HEFT	PRED vs B4	B4 s	PRED s	HEFT s
h16_l120	-5.2%	-17.8%	4.01	3.30	3.48
uniform	-0.2%	+0.0%	2.30	2.30	2.31
h04_l120	-2.4%	-39.4%	3.84	2.33	2.39
h40_l120	-4.8%	-2.8%	6.15	5.98	6.28
h16_l40	-3.2%	-4.8%	2.48	2.36	2.43
h16_l15	-1.4%	-0.7%	2.04	2.03	2.06

GPU findings

- Verified speed-up records: edges 7.7x, features 2.9x, denoise 8 it 1.16x; denoise 2 it 0.99x dropped; 6/15/40/120 it 1.06-1.77x (512 px, incl. H2D/D2H).
- PRED extended set vs CPU only -7.5% [-7.8, -7.2]; strict set vs CPU only -2.6% [-3.3, -1.7] (C9).
- Blind B4 with the GPU: +1.6% [+0.2, +3.0] (C10, inconclusive) - the GPU does not help without a cost model.
- GPU cost model is the weakest part: 23-31% median error on hold-out (CPU 4-11%).
- One CUDA worker; the benefit comes from putting the right tasks on it.

Honest findings

- A hand-written parameter-aware rule (B4_param_first) is within 1% of PRED: most of the gain is parameter-aware choice, not ranking as such.
- The end-to-end cost terms changed nothing: C2 +0.6% [+0.1, +1.3], C7 -0.0% [-0.7, +0.5] (equivalent).
- On the uniform workload the policies tie; PRED_E2E on the strict GPU set was slower than B4 (exploratory).
- Hybrid B4/PRED selector: not justified (best possible gain 1.1% over always using PRED).
- PEFT is clearly the slowest static plan; adding end-to-end terms to HEFT did not help consistently.

Remote worker and limits

- Remote worker: emulated only (loopback node behind a delay shim); on a fast link no benefit, on a slow link blind placement was harmful; not rerun with the final policies.
- One machine, one workflow family, 512 px; confirmatory only for the primary workload; other five are exploratory.
- GPU estimates are rough, so part of the static-vs-dynamic gap on GPU configurations may be model error.
- Static oracle reference uses class medians, not realised per-run times.
- No Raspberry Pi or real-network result.

Contribution

- A reproducible end-to-end timing convention and benchmark that compares static and dynamic scheduling with the same cost model on a CPU+GPU workstation.
- Per-task GPU eligibility derived from recorded speed-up measurements, with the GPU transfer cost counted exactly once.
- Result: the dynamic policy is faster than the static plan by -5.2% (CPU) to -12.2% (strict GPU) on the primary workload, ties blind B4 where task cost does not vary, and a simple parameter-aware rule gets nearly the same gain.
- Everything is logged, hashed and reproducible; failures and superseded files are kept.